Image Deconvolution and Fourier Transform in Image Filtering

Bc. Tomáš Vlach[1]

[1]**xvlach24@stud.fit.vutbr.cz**

ABSTRACT

An image processing project assignment for the courses ZPO and MUL. The project is split into three sections. The first section is about the use of the Fourier transform (its variants) in convolution, image processing and filtering. The second section is about image deconvolution, more specifically about the use of blind deconvolution in guessing the kernel (PSF) that an image has been filtered through (changed by) and reversing the effects to an extend. The third section is combining the two previous sections together into a usable GUI application that can be used for image filtering and restoration.

Keywords: convolution, deconvolution, blind deconvolution, Min-Rank-Kernel deconvolution, Fourier transform, image processing, image filtering

Contents

1 ORGANIZATION DISCLAIMER

This is a solo project fully worked on by *Bc. Tomáš Vlach* (xvlach24). As a result, this paper does not contain any division of work between team members or separation between the team and individual portions of the paper.

2 ASSIGNMENT

The original and revised assignments are as follows.

2.1 Original Assignment

Využití (I)FFT při úpravě a censuře obrazu

Vytvořte program, který za pomocí (I)FFT dokáže upravovat předem určené části obrazu (rozmazání, vyhlazení, atd.) a také dokáže zpětně získat originální obraz. Ukažte že cenzura skrze daný procesy není destruktivní a prozkoumejte implementace

zpětného odhadu kernelu použitého pro rozmazání obrazu.

2.2 Revised Assignment

Create a GUI application that implements two main functionalities.

The first function is image filtering (convolution) within the spectral domain through versions of the Fourier transformation. The application should include options for standard kernels for blurring, high/low pass, smoothing, etc, but also allow the user to create custom kernels.

The second function is blind deconvolution that can estimate the kernel by which an image has been filtered through or by which the camera was affected while capturing the image. Subsequently, use the resulting kernel to estimate the original image through deconvolution methods such as Fast Bregman or others as may suit the project. In addition, the project may include any other image restoration techniques.

If possible, test the results of both functions against already existing solutions in both quality and time.

3 DATA

The data set for this project mainly consists of publicly available images and self-captured images. The convolution section of this project does not require any specific images, but the deconvolution section does.

These pictures will be altered through the convolution method with reasonable kernels (kernels that alter the image but do not completely destroy it) or will be the result of issues during capture, such as motion blur, damaged sensor or optical system, etc. An example of such an image can be seen in figure 1. This specific figure is an example

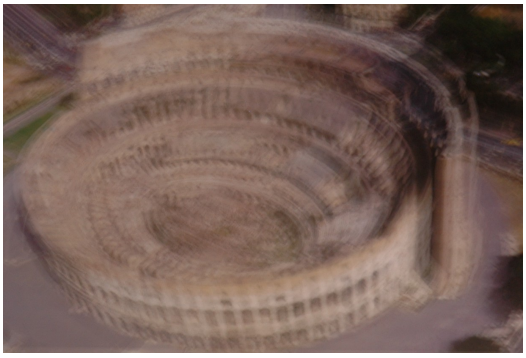

Figure 1. Roma Image Example

from the Min-Rank-Kernel blind deconvolution paper mentioned later.

4 SPECTRAL CONVOLUTION

The spectral convolution will closely resemble already implemented versions of similar systems, such as *fftconvolve* (1) in the *scipy.signals* python library. The aim will be to enable faster convolution without compromising the quality of the result. This also involves including options for *valid* and *same* convolution outcomes, where the resulting size of the convolved image is either smaller due to framing of the edges or the same as the original.

Importantly, the methods in this section will be used during blind deconvolution in the second half of the project. Paradoxically, blind *deconvolution* uses a lot of convolution. During testing, most of the computation time of these methods is spent in convolution (with a runtime of $43s$ it was more than half the time) and accelerating these methods significantly speeds up the deconvolution. The main two points of focus are thus speed and accuracy (the naked eye might not notice small artefacts, but the deconvolution algorithm is rather sensitive).

As sources for further studies and implementation, look, for example, at *The Scien-*

tist and Engineer's Guide to Digital Signal Processing (2) and more.

5 BLIND DECONVOLUTION

For blind convolution I have chosen to implement and work with the *Min-Rank-Kernel blind deconvolution* (MRKd) algorithm described in the article *Understanding Kernel Size in Blind Deconvolution* (3). This algorithm is a version of the MAP blind deconvolution that addresses some of the issues with the said method, most prominently as the paper quotes the *large kernel effect*, which occurs when the size of the output kernel exceeds the original kernel introducing artefacts that degrade the restored image.

This algorithm takes in the altered image, an estimated kernel size and then alternates between optimisation of the expected original image, the kernel, and the rank of the kernel, which minimises the large kernel effect and allows the kernel to outgrow the original kernel size without suffering from artefacts.

In this project the MRKd deconvolution will be used to estimate the kernel and then additional deconvolution algorithms (for example Fast Bregman) will be used to restore the image. In addition, some minor changes, optimisation, and automatic parameter settings will be added to help smooth the process and enhance the experience of working with the algorithm.

5.1 Parameters

There are, for example, several parameters in the MRKd that can be adjusted depending on the target image. These include the parameter τ, which is the sensitivity of rank optimisation, which can be adjusted from $1e-5$ for larger pictures to $3e-4$ for smaller pictures or the number of iterations per cycle (*imax*, *ximax*, *xjmax*, *rmax*) which can be adjusted depending on the sharpness of the original picture. Both of these parameters can be automatically set, depending on the meta data and analysis of the target picture, before the deconvolution starts, if the user chooses to leave these settings in automatic mode (the application will contain the option to manually adjust these).

5.2 Expansion

An optional extension to the project might also be the automatic detection of patterns in the resulting kernel. For example artificially added kernels such as the Gaussian blur have a distinct kernel shape and values, which can be detected and used in further restoration.

The detection system would have to be rather complex, most probably a neural network trained on recognising the kernel patterns, as the blind deconvolution finds both the original kernel and any other imperfections in the original picture, which are combined together into one kernel. This can be seen in figure 2, where the algorithm is finding a Gaussian blur kernel on the left, but the resulting kernel is warped by the addition of the imperfections in the original image (the resulting kernel has double the size for better extraction).

6 DESIGN

The base implementation platform will be *Python 3.12* with additional libraries, notable of which are *OpenCV*, *numpy*, and *tkinter*.

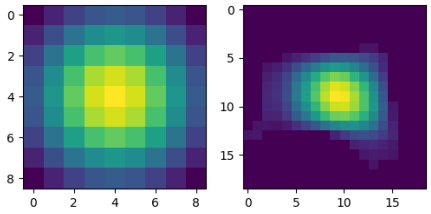

Figure 2. The original kernel (left) and the extracted altered kernel (right). The extracted kernel has double the size for better extraction.

Figure 3. Basic GUI Outline

6.1 GUI and Libraries

The GUI application will be handled through the standard library *tkinter*, for an easy yet multiplatform graphics user interface and basic OS interactions, such as file selection. The interface will be a tabbed document interface that allows simultaneous work on multiple files. The basic outline of the graphics implementation can be seen in figure 3 (an early testing prototype). Additional features missing from this outline and which are planned to be added consist of: a bottom bar displaying image information, additional menus for convolution/deconvolution, image area selection (for working only parts of the original image), and pop-up windows for selecting convolution/deconvolution settings.

Program interactions with image files, such as loading, saving and some conversion, will be done through the *OpenCV* (*opencv-python* version(4)) library. Most of the project essential functions, such as FFT convolution, that are implemented in this library will be avoided, but can be used to compare with the custom solutions.

Most of the mathematical operations will be handled through *numpy*, which is compatible (or default) with other libraries, such as *OpenCV*. Additionally, the *scipy.signals* library will be used to compare and test the spectral convolution results.

6.2 Classes and Structure

The project structure will be broken up into separate files and classes based on their intended use and relation. For better continuity and data handling, several additional support classes will be implemented, such as *CImage* (Custom Image), which will hold image data, metadata, and conversion method between needed formats (RBG Int/Double, BGR Int/Double, YCBCR Int/Double). Individual sections of the project, such as just the spectral convolution, will also have a basic command prompt interface for the option of calling just these sections in separate scripts (such as bash).

The basic outline for the class project structure can be found in figure 4. This is only a rough guideline, as the amount of methods in each class would be overwhelming to put into this diagram. The logical separation of deconvolution classes for example is done for clarity as they contain many methods that overlap in sense and name (like *optimalization of x*), but in

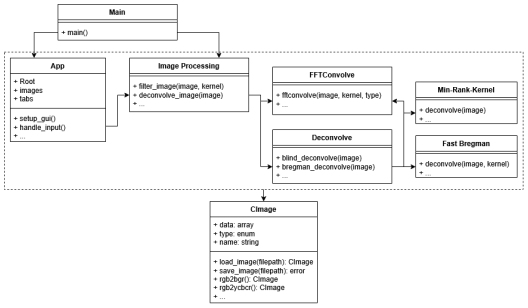

Figure 4. Basic Project Structure

implementation they are crucially different. Additionally, every single class works with the overarching class *CImage* , and so it was separated.

6.3 Testing and Enviroment

The project will also include a Dockerfile, requirements.txt, and bash scripts to create and use a unified test environment. The application will be executable and tested outside this environment on other operating systems and computers.

7 IMPLEMENTATION

Most of the design ideas outlined in the previous sections have been implemented, and the functionalities left out were only possible expansions upon the project base. The implementation is attached within the submission and can be found on Github through this link: https://github.com/AlfieLeFluffy/MUL_ZPO_Project

7.1 Convolution

The spectral convolution implemented within the *SpectralConvolution* class has been split into two main methods. These are *fft_convolve* and *fft_filter* and each is responsible for slightly different image manipulations within the spectral domain.

The first method *fft_convolve* takes both an image and a kernel. Based on the number of dimensions the input image has, it splits the image into layers and runs a single layer *fft_convolve* on each or runs the convolution if the image has only one layer. For the spectral conversion it uses the *numpy.fft.fft2* function for both the image and the kernel with a pre-calculated size based on the size of both inputs. The output of both spectral conversions is then multiplied together, and their result is converted back into normal space using the functions *numpy.fft.ifft2* and *numpy.real*.

The final output of *fft_convolve* is subsequently cut based on the selected mode, which can be either ”same” (same size as the input image), ”full” (the full uncut output of the convolution), or ”valid” (the expected size for normal convolution without padding).

The function *fft_filter* is similar to the previous one, but instead of accepting a kernel, this one requires a type of spectral filter and an offset for a selected filter. These filters are all spectral domain filters, such as a high pass filter, low pass filter, hamming window, which do not have a good normal space representation.

Kernels and filters for both functions are generated dynamically based on the user input through a separate class *KernelGeneration*.

In runtime testing, the method *fft_convolve* outperformed the *scipy.convolve2d*(5) function significantly on larger images, but also lagged behind *scipy.fftconvolve*(6). This is mostly due to the higher level logic of *fft_convolve* being implemented within python, while *scipy.fftconvolve* uses a more effective C++ binding.

Method	Runtime (s)
fft_convolve	~ 2.2
scipy.convolve2d	~ 8.1
scipy.fftconvolve	~ 0.5

Table 1. Convolution runtime compartment (10 cycles on full 825x1229 image with 25x25 kernel)

7.2 Deconvolution

As per the design section, I have decided upon and implemented the Min-Rank-Kernel blind deconvolution (3) and a version of the split fast Bregman non-blind deconvolution. Each of these algorithms are implemented separately for easier use and for the possibility of swapping out of algorithms for different ones.

Currently, the deconvolution chain involves several steps including options forms, but the important ones are: First the Min-Rank-Kernel algorithm takes in the input image, expected kernel size and algorithm parameters, which are sued to calculate the expected kernel. In the second step, the Bregman algorithm takes the original image and the expected kernel and returns an approximation of the unaffected image. The last step is left up to the user, in which the resulting image can be further filtered for better quality using, such as a high pass filter and or a hamming window.

For example of the Min-Rank-Kernel output, in the figure 5 you can see a comparison between the results of the original implementation and this implementation. Both of these images are the result of running the algorithm on the image seen in figure 1 with expected kernel size 85.

Examples of complete image deconvolution results can be found in figures 7 and 8, in which we can see both the result for motion blur in a real photo and the results for a Gaussian blur in a synthetically altered image.

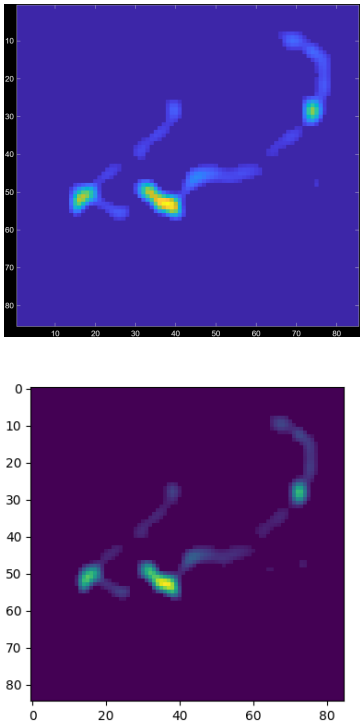

Figure 5. Min-Rank-Kernel comparison between the original (up) and mine (down) on kernel size 85

7.3 Min-Rank-Kernel

The Min-Rank-Kernel implementation focuses on optimisation of several parameters over many cycles. Initially, it builds a list of all kernel sizes through which the algorithm will go and also creates the initial kernel, which is a 3x3 empty kernel with 0.5 at position [0,1] and [1,1].

Each cycle, the algorithm goes through several steps, which include:

- The algorithm resizes the current kernel up to the next size within the kernel size list

- Creates a resized version of the input image based on the current kernel size.
- Both the resized image and kernel are then used to first create and optimise an expected original image.
- These images are used to optimise the kernel.
- The rank of the kernel is optimised.
- Finally, the kernel is re-centred.

Between each kernel size cycle, only the current kernel is saved and everything else is remade/reset for that cycle.

Each optimisation step also involves several cycles, and each cycle usually involves several convolutions, which means that for larger images and expected kernel sizes this can be a rather long process. For example, an 825x1129 image with an expected kernel size of 201 can run 11862 convolutions with a total time of 275 s.

Another aspect of this algorithm is that with larger kernel sizes (getting closer to the end of the size list), the algorithm starts to slow down, scaling down some parameters, and thus taking smaller and more precise steps to maximise the correctness of the final result.

As this algorithm is built on top of another one, expanding it with rank regularisation, it is possible to turn off the kernel rank optimisation during the initial setup and compare the results.

7.4 GUI

During the implementation of the GUI application, a switch was made from native *tkinter* to another library deriving from *tkinter* called *customtkinter*(7), which allows for easier formatting and some expanded functionality. The final UI can be seen in figure 6.

An important feature for the GUI was the parallel execution of code, as the deconvolution algorithms took a substantial amount of time, which would freeze the application without explanation. For this, I have taken the library *Tkinter-Async-Execution*(8), rewrote its *tkinter* implementation into *customtkinter* , and added better *stdout* logging.

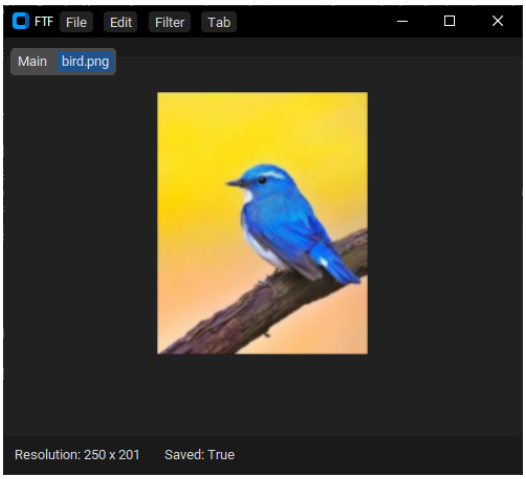

Figure 6. Final look of the GUI application using customtkinter

To adjust the parameters of most functionalities, I have also implemented some form pop-up windows found within the *Form* class, through which the user can change the most important values regarding each kernel/filter/algorithm. These forms are a currently sufficient way of inputting additional data, but if the proposition were expanded, they would most likely have to be rewritten to suit a larger input forms.

There are some OS dependent features with in the GUI, such as the top menu bar being either imbedded in the top bar on Windows or separate on Linux.

8 CONCLUSION

The project implements most of the assigned functionality and in some such as the GUI goes beyond the original expected scope, but there are still things that could make this a much better implementation, and I personally wish I had more time to expand, implement and restructure the project further.

New additions to the project could be additional restoration tools such as new blind or non-blind deconvolution tools, more kernels and better work with them, expanded GUI tools and options, etc. Additionally, the UI and some parts of the project would benefit from further testing, bug fixes, and edge case checks. I am tempted by continuing work on this or a similar project for my Master's thesis.

REFERENCES

[1] T. M. Argument, "fftconvolve — scipy v1.17.0 manual," *Scipy Documentation*, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.fftconvolve.html

[2] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing, 1997, available at www.dspguide.com. [Online]. Available: http://www.dspguide.com

[3] L. Si-Yao, D. Ren, and Q. Yin, "Understanding kernel size in blind deconvolution," *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 2068–2076, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:64672131

[4] Pypi, "opencv-python · pypi," 2026. [Online]. Available: https://pypi.org/project/opencv-python/

[5] Mode, "convolve2d — scipy v1.17.0 manual," 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.convolve2d.html

[6] T. M. Argument, "fftconvolve — scipy v1.17.0 manual," 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.fftconvolve.html

[7] T. Schimansky, "Official documentation and tutorial — customtkinter," 2026. [Online]. Available: https://customtkinter.tomschimansky.com/

[8] D. Hožič, "davidhozic/tkinter-async-execute: A small library, which provides a non-blocking way to run an asyncio event loop ..." 2026. [Online]. Available: https://github.com/davidhozic/Tkinter-Async-Execute

Figure 7. Example of the deconvolution process with top image being original image damaged by motion blur and bottom image is the deconvolved version.

Figure 8. Example of image damaged by low level Gaussian blur and restored using deconvolution. Top is the original, second is the damaged and last is the restored.